Module 11 — Reading: Monitoring, Evaluation, and an Automated Quality Benchmark for a Multi-Service NLP + KG + RAG Stack

(NLP = Natural Language Processing; KG = Knowledge Graph; RAG = Retrieval-Augmented Generation. All three are services you built in Modules 8–10 and revisit this week through the lens of *measurement*.)

> ***Module 10 → Module 11.*** *Module 10 delivered a four-service stack — a FastAPI backend with three endpoints (`/extract`, `/kg/query`, `/rag/answer`), a Next.js client, a Neo4j knowledge graph, and a Weaviate vector store. Module 11 makes that stack tell you whether it is working. Working means three independent claims: the service responds* ***fast*** *enough (latency), it responds* ***at all*** *(error rate), and it responds* ***correctly*** *(answer quality). Each claim is measured differently. This is the last instructional week before the Module 12 capstone, and the two disciplines you build this week — instrumentation (the measurement code) and automated evaluation (the scoring scripts) — are exactly the disciplines the capstone team needs.*

What success looks like: By the end of Module 11, you can add Prometheus counters, histograms, and a `/metrics` endpoint to a FastAPI service, emit structured request-id-correlated logs, read `/metrics` from code with `httpx`, build automated evaluation scripts that score NER (Named Entity Recognition), NL→Cypher, and RAG (Retrieval-Augmented Generation) endpoints against labeled fixtures, and compare latency, error rate, and answer quality as three orthogonal axes of service health.

In This Reading

1. Setup: install before the concept session
2. From deployable to observable — what changes
3. Metrics, logs, and traces — three layers of observability
4. The four Prometheus metric types
5. The OpenMetrics text format — what `/metrics` returns
6. The `prometheus_client` library
7. Histogram bucket selection
8. Choosing labels — bounded label sets only
9. Three middleware layers — request-id, structured logging, metrics
10. Reading `/metrics` from code
11. The evaluation harness shape
12. NER F1 methodology
13. NL→Cypher exact-match methodology
14. RAG grounding-rate methodology
15. Three axes of service health — fast, reliable, correct
16. Monolith and microservice — revisited from Module 10
17. Distributed observability — a preview
18. What `/metrics` cannot tell you

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Setup: install before the concept session

Module 11 introduces **one new Python package** on top of the Module 10 environment. Everything else — Docker Desktop, Node.js, the four-service image cache (`api`, `web`, `neo4j`, `weaviate`) — is reused unchanged from Module 10.

> ***Hardware-profile note.*** *The same 16 GB RAM laptop that ran the Module 10 stack runs Module 11. Module 11 reuses the Module 10 `flan-t5-base` generator (the one large in-memory artifact, loaded once in `lifespan` exactly as in Module 10) and reuses the Module 10 Neo4j and Weaviate services unchanged.*

1.1 Install `prometheus-client`

```
pip install prometheus-client
```

The Lab and Integration template repos both include a `requirements.txt` that pins this package — running `pip install -r requirements.txt` after you fork the template repos adds it automatically. The reading installs it directly so you can run the code examples below in your existing venv.

Verify:

```
python -c "import httpx, prometheus_client; print('ok')"
```

`httpx` already installed from Module 10; if the verify line raises `ModuleNotFoundError: httpx`, run `pip install httpx` to repair the Module 10 environment before continuing.

1.2 Same install command on every platform

The `prometheus_client` package is pure Python with no compiled extensions, so the same `pip install prometheus-client` command works on every platform you are likely to use: macOS, Linux, Windows (PowerShell, CMD (Command Prompt), WSL2 (Windows Subsystem for Linux 2), and Git Bash).

1.3 Reusing the Module 10 stack

The Lab and Integration both run against the four-service Docker Compose stack you built in Module 10. The Lab template repo includes a copy of the Module 10 backend (`api/`, `web/`, `docker-compose.yml`, `seed_neo4j.sh`, `seed_weaviate.sh`) so a learner who is still finishing the Module 10 Lab can still complete Module 11. The pip cache from Module 10 keeps all later installs fast.

---

2. From deployable to observable — what changes

Module 10 delivered a stack. The four services start with `docker compose up -d`, the API (application programming interface) answers requests, the client renders results, the knowledge graph and the vector store hold their fixtures. That is *built*. It is not yet *observable*.

Observability is the property that lets an operator — or a downstream automated client, or an alerting system, or you, three months from now, paged at 3 a.m. — answer the question *"is this service working right now?"* without reading the source code. A service that answers requests but cannot tell you whether it is healthy is a service that fails silently. Module 11 adds the surfaces that let the service speak.

There are **three claims** packed into "working":

1. **Fast.** The service responds within an acceptable latency budget. Quantified by percentile latencies (p50, p95, p99) over a sliding window.
2. **Reliable.** The service responds without errors most of the time. Quantified by error rate — the fraction of requests that return a 5xx (server-side failure) status.
3. **Correct.** The service responds with the *right answer*. Quantified by a domain-specific quality metric — for NER, token-level F1; for NL→Cypher, exact-match accuracy; for RAG, the *grounding rate* (the fraction of answers whose citations all point to chunks the retrieval call actually returned, so the model is not making up sources).

These three claims are **independent**. A service can be fast and reliable and wrong. A service can be slow and correct. A service can be fast and correct and crash 10% of the time. Each axis requires its own measurement. The two disciplines this week are (i) **instrumentation** — the code that exposes latency and error rate via a `/metrics` endpoint — and (ii) **evaluation** — the script that measures correctness against a labeled fixture. The reading frames the three axes up front and returns to them in §15.

---

3. Metrics, logs, and traces — three layers of observability

Three different *kinds* of observability data, each with a different shape and a different cost:

- **Metrics.** Pre-aggregated numerical time series. One counter increments per request; one histogram bucket fills per latency observation. A metric is *small* — bytes per minute regardless of traffic. Read by humans (dashboards) and machines (alerting rules). Module 11 covers metrics in depth.
- **Logs.** Per-event records. One JSON line per request with `request_id`, `path`, `status`, `latency_ms`, and any error detail. A log is *large* — bytes per request. Read by humans (debugging) and machines (search, alerting). Module 11 covers logs in depth — specifically, structured (JSON) logs correlated by `request_id`.
- **Traces.** Causal chains across services. One trace per request, with one span per service hop. Traces are large and require a tracing backend (Jaeger, Tempo, an OpenTelemetry collector). Module 11 covers traces as a *concept* but does not implement them — distributed tracing is the Stretch (Honors Track) surface for learners who go deeper.

You cannot derive metrics from logs cheaply at high volume — that is why pre-aggregation exists. You cannot derive per-request detail from metrics — that is why logs exist. Production services typically want all three; this module covers the two that the Module 12 capstone requires.

---

4. The four Prometheus metric types

Prometheus is the standard format for service metrics in modern Python (and Go, and most other backend languages). The `prometheus_client` Python library implements four metric types; you choose the one that fits the signal you are measuring.

- **Counter.** A monotonically increasing value. Once it goes up, it does not come down (except at process restart). Use for **request counts**, **error counts**, **bytes processed**. Wrong choice for *current in-flight requests* — that value goes down.
- **Gauge.** A value that can go up *or* down. Use for **in-flight requests**, **last-readiness-check timestamp**, **current memory use**. Wrong choice for *total requests* — a gauge can decrease, which makes derived signals like "requests per second" undefined.
- **Histogram.** A bucketed distribution — a row of counters, one per latency range. A *bucket* in Prometheus is a counter that increments every time an observation is less than or equal to its upper bound. Each observation increments the bucket it falls into and the `_count` + `_sum` aggregates. Use for **request latency** (the canonical case), **response size**, anything where you want percentile signals after the fact. Bucket boundaries are part of the design — see §7.
- **Summary.** Like a histogram, but quantiles are computed client-side. Not used in this module. Histograms are preferred because their bucket counts are *aggregatable* across processes — summary quantiles are not.

What breaks when you pick the wrong type: if you use a gauge for request counts, every `request_count.set(...)` overwrites the previous value, so the metric only shows the *last* increment. The reading shows this exact mistake on a hello-world FastAPI service in §9.

---

5. The OpenMetrics text format — what `/metrics` returns

The `/metrics` endpoint serves plain text in the OpenMetrics format. Anyone with `curl` can read it. Here is a 12-line excerpt from a hello-world service that has handled three `POST /echo` requests:

```
# HELP requests_total Total HTTP requests
# TYPE requests_total counter
requests_total{path="/echo",status="200"} 3.0
# HELP request_latency_seconds Request latency in seconds
# TYPE request_latency_seconds histogram
request_latency_seconds_bucket{path="/echo",le="0.005"} 0.0
request_latency_seconds_bucket{path="/echo",le="0.01"} 2.0
request_latency_seconds_bucket{path="/echo",le="0.025"} 3.0
request_latency_seconds_bucket{path="/echo",le="+Inf"} 3.0
request_latency_seconds_count{path="/echo"} 3.0
request_latency_seconds_sum{path="/echo"} 0.018
# HELP inflight_requests In-flight requests
# TYPE inflight_requests gauge
inflight_requests 0.0
```

The format is intentionally readable. Three pieces are worth naming:

- **`# HELP` and `# TYPE` comment lines** describe each metric — these are how a Prometheus scraper (or your evaluation script) learns what kind of metric each name represents.
- **One sample line per (metric, label-set) tuple.** `requests_total{path="/echo",status="200"} 3.0` reads as "the counter `requests_total` with labels `path="/echo"` and `status="200"` has value 3.0." If a fourth request returns a 500, a new line appears: `requests_total{path="/echo",status="500"} 1.0`.
- **Histograms emit three derived lines per (metric, label-set):** `_bucket` lines for each cumulative bucket boundary `le=...` (less-than-or-equal), a `_count` line for total observations, and a `_sum` line for the sum of observed values. The buckets are *cumulative* — `_bucket{le="0.025"} 3.0` means three observations had latency ≤ 25 ms (which includes the two that had latency ≤ 10 ms).

The `+Inf` bucket is always present — it catches observations larger than the largest finite bucket. If `_bucket{le="+Inf"}` differs from `_count`, your scraper is wrong.

The excerpt above is trimmed for readability. When you `curl /metrics` against a real `prometheus_client` service you will also see one `_created` line per `Counter` and per `Histogram` — for example `requests_created{path="/echo",status="200"} 1.706e+09` and `request_latency_seconds_created{path="/echo"} 1.706e+09`. These record the Unix timestamp at which the metric was first registered. They are part of the OpenMetrics spec and are not a sign of a bug — see Troubleshooting if your output looks unexpectedly long.

---

6. The `prometheus_client` library

The Python library that implements the OpenMetrics format. The four metric types you will use this week:

```python
from prometheus_client import Counter, Histogram, Gauge, make_asgi_app

# Counter
requests_total = Counter(
    "requests_total",
    "Total HTTP requests",
    ["path", "status"],
)
requests_total.labels(path="/extract", status="200").inc()

# Histogram with explicit buckets
request_latency_seconds = Histogram(
    "request_latency_seconds",
    "Request latency in seconds",
    ["path"],
    buckets=[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
)
request_latency_seconds.labels(path="/extract").observe(0.012)

# Gauge
inflight_requests = Gauge("inflight_requests", "In-flight requests")
inflight_requests.inc()    # entering a request
inflight_requests.dec()    # exiting a request

# Expose at /metrics on a FastAPI app
from fastapi import FastAPI
app = FastAPI()
app.mount("/metrics", make_asgi_app())
```

The `Counter`, `Histogram`, and `Gauge` declarations are *module-level* — declare them once at import time, then call `.labels(...).inc()` or `.observe(value)` from the middleware that records each request. `make_asgi_app()` returns an ASGI (asynchronous server gateway interface) app whose only purpose is to serve the current state of all registered metrics in the OpenMetrics format; `app.mount(...)` attaches it at a path of your choosing (convention: `/metrics`).

There is one subtlety. The `.labels(path=..., status=...)` call returns a *child* of the parent metric — a labelled timeseries. If you call `.inc()` on the parent (`requests_total.inc()`), `prometheus_client` raises an error because the parent has labels declared. Always `.labels(...).inc()`, never bare `.inc()`, when labels are declared.

---

7. Histogram bucket selection

Buckets are part of the design. The default buckets for `request_latency_seconds`:

```
buckets = [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10]
```

These span 5 ms to 10 s. They are *unevenly spaced* — the resolution is fine at the short-latency end and coarse at the long-latency end. The Module 10 stack has three endpoint shapes with very different latency profiles:

- `/extract` — spaCy NER, ~10 ms hot. Falls in the 5–25 ms region.
- `/kg/query` — deterministic Cypher mapper + Neo4j round-trip, ~50 ms. Falls in the 25–100 ms region.
- `/rag/answer` — embedding + Weaviate retrieval + `flan-t5-base` generation, ~1–3 s on local model. Falls in the 0.5–5 s region.

The default buckets cover all three with reasonable resolution. **Resolution matters for percentile estimation.** When your p95 falls between two bucket boundaries, the value is interpolated; a wide bucket spreads the estimate over a wide range. If your largest bucket is `0.1` and your p95 is actually 1.2 s, every observation falls into the `+Inf` bucket and your p95 is reported as `+Inf` — uninformative.

Design rule. Size buckets for the actual distribution, not the hoped-for distribution. If you do not know the distribution yet, observe a few hundred requests with the default buckets, look at the bucket counts, and refine. The Lab Challenge Tier 1 walks through this exact exercise for the `/rag/answer` endpoint.

---

8. Choosing labels — bounded label sets only

Each combination of label values produces one separate timeseries in `/metrics`. Few timeseries per metric is good; many timeseries per metric is a bug. The number of timeseries a metric produces grows with the size of each label's value set, so the rule for choosing labels is: only use a label when its value set is **bounded and small** (you can list every possible value). The rest of this section walks through why.

Labels are **partitions of a discrete known set**. A counter with labels `["path", "status"]` produces one timeseries per (path, status) combination — for the Module 10 stack, that is about five paths × five status families ≈ 25 timeseries. That is fine.

A counter with a label `user_id` produces one timeseries per *user* — for a service with 10,000 users, that is 10,000 timeseries. That is **a bug**, not a feature. Three things break:

- **Memory.** Every active timeseries holds counter state in process memory. 10,000 timeseries is roughly tens of megabytes of `prometheus_client` overhead.
- **Scrape size.** Every active timeseries emits at least one sample line on `/metrics`. A scrape that returns 10,000 lines saturates network and parser budgets.
- **Aggregation.** Prometheus queries over metrics with many distinct timeseries are slow and often timeout.

The design rule is **labels are partitions of a discrete known set**. `path` is bounded (you know all your routes). `status` is bounded (200, 400, 422, 500 — about five values). `method` is bounded (GET, POST, PUT, DELETE). `user_id`, `request_id`, `raw_input_text`, anything unbounded — **never as a metric label**. Request-level detail belongs in *logs* and *traces*, not in metric labels.

---

9. Three middleware layers — request-id, structured logging, metrics

The instrumentation that the Lab asks you to add is three middleware layers wired in the right order. Worked example for a FastAPI app:

```python
import contextvars, json, logging, time, uuid
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, Gauge, make_asgi_app
from starlette.middleware.base import BaseHTTPMiddleware

request_id_var: contextvars.ContextVar[str] = contextvars.ContextVar("request_id"

requests_total = Counter("requests_total", "Total HTTP requests", ["path", "statu
request_latency_seconds = Histogram(
    "request_latency_seconds",
    "Request latency in seconds",
    ["path"],
    buckets=[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
)
inflight_requests = Gauge("inflight_requests", "In-flight requests")


class RequestIdMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        rid = uuid.uuid4().hex
        token = request_id_var.set(rid)
        try:
            response = await call_next(request)
        finally:
            request_id_var.reset(token)
        response.headers["X-Request-ID"] = rid
        return response


class StructuredLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        started = time.perf_counter()
        response = await call_next(request)
        latency_ms = (time.perf_counter() - started) * 1000
        log_line = json.dumps({
            "ts": time.time(),
            "level": "INFO",
            "request_id": request_id_var.get(),
            "path": request.url.path,
            "status": response.status_code,
            "latency_ms": round(latency_ms, 2),
        })
        logging.getLogger("app").info(log_line)
        return response


class MetricsMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        inflight_requests.inc()
        started = time.perf_counter()
        try:
            response = await call_next(request)
        finally:
            inflight_requests.dec()
        latency = time.perf_counter() - started
        path = request.url.path
        requests_total.labels(path=path, status=str(response.status_code)).inc()
        request_latency_seconds.labels(path=path).observe(latency)
        return response


app = FastAPI()
# Outermost: request-id. Innermost (closest to the route): metrics.
app.add_middleware(MetricsMiddleware)
app.add_middleware(StructuredLoggingMiddleware)
app.add_middleware(RequestIdMiddleware)
app.mount("/metrics", make_asgi_app())
```

Two points are important:

Order matters. Starlette runs `add_middleware` calls in *reverse* order — the last `add_middleware` runs first on the way in. So in the code above, the request first hits `RequestIdMiddleware` (sets the id), then `StructuredLoggingMiddleware` (will read the id on the way out), then `MetricsMiddleware` (closest to the route, times the actual handler). If you flip metrics outermost, the timing includes the time spent in the other two middlewares — which is small, but the design intent is "time the handler, not the middleware chain."

The `X-Request-ID` response header. The same `request_id` that appears in the structured log appears in the response header. A client (or load tester, or downstream service) can capture the header and correlate a failure they observed with the log line that recorded the request server-side. This is the cheapest *distributed* observability you can get without a tracing system.

---

10. Reading `/metrics` from code

The Module 11 Integration evaluation script reads `/metrics` directly via `httpx` and extracts derived signals — per-endpoint p95 latency, per-endpoint error rate, total request counts. The `prometheus_client.parser` module provides a text parser:

```python
import httpx
from prometheus_client.parser import text_string_to_metric_families

response = httpx.get("http://localhost:8000/metrics", timeout=5.0)
response.raise_for_status()
families = list(text_string_to_metric_families(response.text))

for family in families:
    if family.name == "requests":
        for sample in family.samples:
            if sample.name == "requests_total":
                print(sample.name, sample.labels, sample.value)
```

Each `MetricFamily` has `.name`, `.type`, and `.samples`. Each sample is a `(name, labels, value)` triple. For a counter, the samples are the single timeseries per label-set. For a histogram, the samples include `_bucket` lines, a `_count` line, and a `_sum` line per label-set.

10.1 Computing error rate from counters

Error rate per path = (sum of counter samples whose status starts with "5") / (sum of all counter samples for that path).

```python
def error_rate(samples, path):
    errors = sum(
        s.value for s in samples
        if s.labels.get("path") == path and s.labels.get("status", "").startswith
    )
    total = sum(
        s.value for s in samples
        if s.labels.get("path") == path
    )
    return errors / total if total > 0 else 0.0
```

10.2 Computing p95 latency from histogram buckets

Histograms report *cumulative* bucket counts. To compute p95: find the bucket whose cumulative count first crosses `0.95 * total_count`, then linearly interpolate within that bucket.

```python
def p95_latency(family, path):
    # Collect this path's bucket samples and the total count.
    buckets = []
    total = 0.0
    for sample in family.samples:
        if sample.labels.get("path") != path:
            continue
        if sample.name.endswith("_bucket"):
            le = float(sample.labels["le"]) if sample.labels["le"] != "+Inf" else
            buckets.append((le, sample.value))
        elif sample.name.endswith("_count"):
            total = sample.value
    buckets.sort()
    target = 0.95 * total
    prev_le, prev_count = 0.0, 0.0
    for le, count in buckets:
        if count >= target:
            # Linear interpolation within the (prev_le, le) bucket.
            if count == prev_count:
                return le
            fraction = (target - prev_count) / (count - prev_count)
            if le == float("inf"):
                # +Inf bucket — no upper bound, so we cannot estimate the
                # actual latency. Return +Inf so the signal is obvious in
                # the report; the caller (or the dashboard) should treat
                # this as "your buckets are too small, fix them".
                return float("inf")
            return prev_le + fraction * (le - prev_le)
        prev_le, prev_count = le, count
    return 0.0
```

When the p95 target falls in the `+Inf` bucket, you have no upper bound — the function returns `+Inf`, which is a signal that your buckets are wrong (see Troubleshooting). Do not silently substitute the previous finite boundary: that masks the very signal the histogram was supposed to give you.

---

11. The evaluation harness shape

`/metrics` reports *fast* and *reliable*. It cannot report *correct*. Correctness requires a labeled fixture, a methodology, and a script that runs against the live service. The harness is a *script*, not a notebook — the service is what gets delivered to users, the service is what gets evaluated, and the harness must be reproducible from one command.

The shape, in five steps:

1. **Load the fixture.** A JSON or YAML file of (input, gold-answer) pairs.
2. **Iterate.** For each input, issue `httpx.post(url + endpoint, json={...})`. Capture the response.
3. **Score.** Apply the endpoint-specific metric: NER F1 (§12), exact-match (§13), grounding rate (§14).
4. **Accumulate.** Per-question results in a list; a running aggregate of TPs/FPs/FNs or correct/incorrect counts.
5. **Write the report.** A Markdown file with one section per endpoint: headline number, the methodology paragraph (stated identically in spec, guide, and docstring per the Evaluation Methodology Rule), a per-question table, sample failures with gold vs. predicted side-by-side.

The Integration provides three harnesses (`eval_ner.py`, `eval_kg.py`, `eval_rag.py`) following this exact shape, plus a small orchestrator (`eval_runner.py`) that calls all three, reads `/metrics`, and assembles the consolidated report.

---

12. NER F1 methodology

For the `/extract` endpoint. Token-level entity match, micro-averaged across documents.

Definitions. An *entity* is a tuple `(document_id, entity_text, entity_label)`. A predicted entity is a *true positive* (TP) if and only if a gold entity with the **same `document_id`, the same `entity_text`, and the same `entity_label`** exists in the gold set. A predicted entity that has no matching gold entity is a *false positive* (FP). A gold entity that has no matching predicted entity is a *false negative* (FN).

Precision. `TP / (TP + FP)`. Of the entities the model predicted, what fraction were correct?

Recall. `TP / (TP + FN)`. Of the entities that should have been predicted, what fraction did the model find?

F1. `2 * P * R / (P + R)`. Harmonic mean of precision and recall.

Micro-averaging. Sum TPs, FPs, FNs across *all* documents, then compute precision / recall / F1 once on the totals. The alternative (*macro-averaging* — compute F1 per document, then average the F1s) weights every document equally, which is the wrong choice when document length varies; an entity-dense document and an entity-sparse document should not contribute equally to the headline. Micro-averaging is entity-density-weighted.

Filtering rule. The evaluation set vendored in the Integration is a 30-document subset of the CoNLL-2003 development split. The harness scores only documents whose `document_id` appears in the gold fixture — if the API returns extractions for documents the harness did not ask about, they are not scored (they cannot be — there is no gold to compare against).

What gets checked. The Integration's pre-built backend is fixture-driven (it echoes the gold entities for fixture-matched inputs), so a correctly-implemented harness sees F1 ≈ 1.0 against this backend. The CI (continuous integration) check is sensitive to broken harness implementations (wrong micro-aggregation, off-by-one TP counting); it is not a gate on model quality.

Label-scheme caveat. This methodology compares `entity_label` strings with exact equality. The gold fixture uses CoNLL-2003 labels (`PER`, `LOC`, `ORG`, `MISC`). If you later point the harness at a real `en_core_web_sm` backend, those predictions arrive in the OntoNotes-5 scheme (`PERSON`, `GPE`, `ORG`, `NORP`, …) and the harness needs a remap layer before comparing. See Troubleshooting.

---

13. NL→Cypher exact-match methodology

For the `/kg/query` endpoint. The endpoint is backed by the deterministic NL→Cypher mapper you carried forward from Module 9 Week B — it produces the same output for the same input, by construction, on the bounded schema. That is what makes exact-match a meaningful metric: there is one correct Cypher per question, and the mapper either produces it or it does not.

Normalization. Before comparing the generated Cypher to the gold Cypher, both are normalized:

```python
import re

CYPHER_KEYWORDS = ["MATCH", "RETURN", "WHERE", "OPTIONAL", "WITH", "LIMIT", "ORDE

def normalize_cypher(s: str) -> str:
    # Collapse whitespace.
    s = re.sub(r"\s+", " ", s).strip()
    # Uppercase Cypher keywords.
    for kw in CYPHER_KEYWORDS:
        s = re.sub(rf"\b{kw}\b", kw, s, flags=re.IGNORECASE)
    return s
```

Exact-match. Two normalized strings are equal iff every character matches. No BLEU (Bilingual Evaluation Understudy), no ROUGE (Recall-Oriented Understudy for Gisting Evaluation), no semantic-similarity — those are the right metrics when generation is open-ended and many surface forms are acceptable; here the mapper is bounded, every question has exactly one correct Cypher, and exact-match is the cleanest signal.

Exclusion rule. When the mapper recognizes that a question is outside its supported intent set, the endpoint returns HTTP 422 with `UnsupportedQueryError`. Those questions are excluded from the *denominator* — the metric is *exact-match accuracy on supported questions*, not *exact-match on the full set*. The evaluation set is constructed so most questions are supported; about three of the 25 are deliberately out-of-scope to exercise the decline path.

What gets checked. Normalized exact-match accuracy on the supported subset of the fixture. The Integration's pre-built backend returns the gold Cypher for fixture-matched questions, so a correct harness sees accuracy ≈ 1.0.

---

14. RAG grounding-rate methodology

For the `/rag/answer` endpoint. The metric measures whether the answer is *grounded* in the retrieved passages — whether every claim cites a chunk that was actually returned by the retrieval call.

Grounded. A response is *grounded* if and only if all three conditions hold:

1. The response's `citations` array has length ≥ 1.
2. Every `chunk_id` in the `citations` array appears in the *candidate set* — the chunks that the retrieval call returned for this question with the same `k`. The candidate set is read from the `/rag/answer` response body itself: the endpoint returns a `retrieved` field whose entries each carry a `chunk_id`, so the harness does not re-issue a separate retrieval call.
3. The response is not the canonical *decline* string — the M10 contract specifies that when the retriever returns no relevant chunks, the endpoint must answer `"I cannot answer this from the available sources."` instead of fabricating.

Lab vs. Integration split. The Lab applies the first two conditions only (citations ≥ 1 AND every cited `chunk_id` in the candidate set); decline-exclusion is the Integration's responsibility, where the eval harness sees many questions including declines and must exclude them from the grounding-rate denominator. The reading states all three conditions because the full methodology is what the Integration scales up to; the Lab is the simplified single-stack subset.

Decline exclusion. Declines are excluded from the *denominator*. The metric is grounding rate *on answered questions*, not on the full fixture. The rationale is symmetric to the M10 design intent: a service that *declines* when it cannot answer is behaving correctly; penalizing the decline path would push learners toward fabricating citations to keep the metric high. The metric rewards "answer with citations grounded in retrieval" and is indifferent to "decline when you should."

Computation.

```python
grounded = 0
answered = 0
for question, response in zip(fixture, responses):
    if response["answer"].strip() == "I cannot answer this from the available sou
        continue    # excluded
    answered += 1
    candidate_ids = {entry["chunk_id"] for entry in response.get("retrieved", [])
    cites = response.get("citations", [])
    if len(cites) >= 1 and all(c["chunk_id"] in candidate_ids for c in cites):
        grounded += 1

grounding_rate = grounded / answered if answered else 0.0
```

What gets checked. Grounding rate computed as `grounded / answered` against the Integration's pre-built backend, which returns gold citations resolving to the gold retrieved chunk_ids — a correctly-implemented harness sees grounding rate ≈ 1.0.

---

15. Three axes of service health — fast, reliable, correct

Bring it together. The three axes are independent, and each requires its own measurement:

Axis	What it tells you	How you measure it
Fast	Latency budget	Histogram, p95 from `/metrics`
Reliable	Error rate	Counter ratios from `/metrics`
Correct	Answer quality	Evaluation script against a labeled fixture

A service can be fast and reliable and wrong. (The classic case: a retrieval-augmented generator with a broken grounding check that fabricates citations. Latency is fine; error rate is fine; answers are wrong. The Integration evaluation script catches this.)

A service can be slow and reliable and correct. (Latency tuning is a deployment concern: cache the embedding model, batch the retrieval, run the generator on a hotter machine. The instrumentation tells you *where* to optimize. The evaluation tells you the optimization did not break correctness.)

A service can be fast and correct and crash 10% of the time. (Error budget concern: the failing 10% may be a flaky downstream, a connection pool that needs tuning, a missing retry. The counter ratio tells you the *what*; the structured logs correlated by `request_id` tell you the *why*.)

The three are orthogonal. None of them substitutes for any other. The reading frames this here because the Module 12 capstone item *"Basic dashboard or script logging latency, error rates, and answer quality"* is exactly the three axes — and the capstone team needs every member to have built the harnesses at least once.

---

16. Monolith and microservice — revisited from Module 10

Module 10 introduced the monolith vs microservice trade-off — at the time, abstractly. The Module 10 `api` service is a *monolith* with three endpoints: `/extract` (NER), `/kg/query` (KG), `/rag/answer` (RAG). One container, three responsibilities.

A *microservice* decomposition would split this into three services — a `ner` service, a `kg` service, a `rag` service — each behind its own URL with its own `/metrics` and its own deploy pipeline.

What you get:

- **Independent scaling.** If RAG traffic increases sharply and NER traffic is flat, only the `rag` service needs more replicas.
- **Independent failure domains.** If the embedding model runs out of memory (OOM) in the `rag` service, `/extract` and `/kg/query` keep working.
- **Independent dependency management.** The `rag` service pins `transformers`, `sentence-transformers`, `weaviate-client`; the `ner` service pins `spacy`. Smaller images, faster cold starts.

What you pay:

- **Network latency between services.** A request that previously called three local functions now makes three HTTP calls. p50 latency goes up.
- **Operational complexity.** Three deploy pipelines, three sets of logs, three `/metrics` endpoints, three configuration files. The instrumentation layer you just built has to be replicated.
- **Contract management.** Every change to an internal API now has two consumers (its caller and its tests). Versioning, breaking-change reviews, deprecation periods — all the things you skip in a monolith.

There is no universal answer. The decision depends on traffic patterns, team size, and the cost of an incident. The single-team service handling 10 requests per second probably should stay a monolith. The 50-team service handling 50,000 requests per second probably should not. The capstone team picks a shape; the reading is providing the framing.

---

17. Distributed observability — a preview

If you do decompose into three services, observability multiplies. Each service exposes its own `/metrics`. Each service emits its own structured logs. A single client request now hits three services in sequence — and you need to follow that request as it moves through them.

The pattern: the **request-id** that the outermost service generates flows to every downstream call as a header (`X-Request-ID`). Every downstream service reads the header instead of generating its own. Every log line at every layer carries the same id. Now grep the logs across three services for one id and you have the per-request causal chain by hand.

The platform version of this is **distributed tracing** — OpenTelemetry, Jaeger, Tempo. A tracing client wraps every HTTP call; each call emits a *span* with the request-id (now called a *trace-id*), a *span-id*, timestamps, and tags. A tracing backend assembles the spans into a tree per request.

This module covers the *concept* — request-id propagation across boundaries. Implementing the full distributed-observability pattern across three replica services is the Stretch (Honors Track) assignment for Module 11.

---

18. What `/metrics` cannot tell you

It cannot tell you that the answer is wrong.

A service can have green `/metrics` — low p95, zero 5xx — and produce nonsense. Latency tells you *how long* the response took; counters tell you *whether* the response came back; neither tells you *what* the response said. Quality is the third axis, and `/metrics` does not measure it.

This is why the evaluation script is required alongside the metrics endpoint. The two together — instrumentation for fast and reliable, evaluation for correct — are the three-axis observability frame that the capstone deliverable codifies.

The contrapositive matters too: a service whose evaluation harness reports an excellent grounding rate but whose `/metrics` shows a 50% 5xx error rate is not a *better* service than one with green `/metrics` and a poor grounding rate. Both are broken in different ways. Both axes have to be green for the service to be shippable.

The capstone item 4 — *"Automated script to benchmark retrieval accuracy and generative output grounding"* — is the M11 Integration deliverable. The capstone item 3 — *"Basic dashboard or script logging latency, error rates, and answer quality"* — is the consolidated evaluation report the M11 Integration writes. The skills are introduced this week so the capstone team can divide the work assuming everyone has built each piece at least once.

Troubleshooting

- **`/metrics` returns a FastAPI 404 page.** You forgot the `app.mount("/metrics", make_asgi_app())` line, or mounted at the wrong path. Confirm `make_asgi_app()` is imported from `prometheus_client` (not from `starlette`).
- **`/metrics` returns an empty body except for the comment lines.** The metric declarations live in a module that the worker process did not import. Move the declarations into the module that defines `app`, or ensure the middleware module is imported at app construction time.
- **`/metrics` has unexpected `*_created` lines I do not see in the reading's excerpt.** Expected. `prometheus_client` emits one `requests_created` and one `request_latency_seconds_created` line per `Counter` and `Histogram` (and one per label set). They record the Unix timestamp at which each timeseries was first registered and are part of the OpenMetrics spec. Your scraper should ignore them; you do not need to suppress them.
- **`requests_total` does not increment after a request.** Middleware order is wrong (the metrics middleware came outside the response generation) or you forgot `app.add_middleware(MetricsMiddleware)`. Hit one request, then read `/metrics` and look for a `requests_total` line — if no line at all, the middleware never ran; if a line with value 0, the middleware ran but skipped the `.inc()` (check the path filter).
- **`request_latency_seconds_bucket{...,le="+Inf"} 7.0` while every other bucket is 0.** All your observations exceeded the largest finite bucket. Add larger buckets to the `Histogram` declaration (e.g., extend to 30, 60, 120 seconds for slow endpoints) or check why the endpoint is so slow.
- **`X-Request-ID` header is missing from responses.** The request-id middleware never ran. Confirm it was added with `app.add_middleware(RequestIdMiddleware)` and that no other middleware swallows headers.
- **Structured log lines print as Python objects, not JSON.** You configured the formatter on the root logger but read from a child logger that has its own handler. Configure `logging.basicConfig(...)` with the JSON formatter, or set the formatter on the handler attached to the specific logger.
- **`prometheus_client.parser.text_string_to_metric_families` raises `ValueError` on `+Inf`.** You are on an old version. Upgrade `prometheus-client` to the version in `requirements.txt`.
- **`httpx.get("/metrics")` raises `ReadTimeout`.** The default `httpx` timeout is 5 s; if the service is hanging on `/metrics`, the histogram is probably producing too many distinct timeseries (see §8). Drop the unbounded label and re-test before you raise the timeout.
- **Evaluation harness hangs.** Default `httpx` timeout is 5 s; `/rag/answer` often takes 1–3 s on cold cache and can exceed 5 s on the first request. Pass `httpx.post(url, json=..., timeout=60.0)`.
- **Evaluation report is empty.** The harness silently failed on the first fixture item — usually an `httpx.HTTPStatusError` that the script caught without logging. Add a `--verbose` flag that prints every request/response pair so you can see the first failure.
- **Grounding rate counts declines as failures.** Decline-exclusion not applied — re-read §14. The denominator must be *answered*, not *total*.
- **Cypher exact-match fails on whitespace.** Normalization missing — re-read §13. Always normalize *both* generated and gold before comparing.
- **NER F1 lower than the baseline.** Label vocabulary mismatch. The Integration's gold fixture uses CoNLL-2003 labels (`PER`, `LOC`, `ORG`, `MISC`), but `en_core_web_sm` uses the OntoNotes-5 label scheme and emits `PERSON`, `GPE` (cities/countries), `LOC` (geographic features), `ORG`, `NORP` (nationalities/religions), and others. Direct exact-label equality between OntoNotes predictions and CoNLL gold collapses F1 because nearly every label string differs. If you point the harness at a real spaCy model rather than the fixture-driven Integration backend, you need a remap layer — e.g. `PERSON → PER`, `GPE → LOC`, `LOC → LOC`, `ORG → ORG`, drop the rest — before comparing.

Key Terms

- **Observability** — the property of a service that allows its operators to understand its internal state from its external surfaces.
- **Metrics** — pre-aggregated numerical time series. Counter, Gauge, Histogram, Summary.
- **Logs** — per-event records. Structured (JSON) or unstructured (text).
- **Traces** — causal chains across services. Composed of spans.
- **OpenMetrics** — the standard text format for Prometheus-compatible metric exposition.
- **Prometheus** — the open-source metric collection and query system. This module uses the *client library* (which exposes `/metrics`) without running the Prometheus *server*.
- **Scrape** — a Prometheus server's GET to a service's `/metrics` endpoint at a regular interval. In Module 11 the *evaluation script* plays the role of the scraper.
- **Counter** — a monotonically increasing metric.
- **Gauge** — a metric that can go up or down.
- **Histogram** — a bucketed distribution metric.
- **Summary** — a client-side quantile-computing metric (not used this module).
- **Label** — a key/value tag on a metric. Partitions a metric into multiple timeseries.
- **Timeseries count** — the number of distinct timeseries a metric produces, one per distinct combination of label values. Keeping this small is the goal of §8's label-choosing rule. (The industry term for this number is **cardinality** — you will hear it in production on-call settings; we use plain language here for clarity.)
- **p50, p95, p99** — the 50th, 95th, 99th percentile of an observation distribution.
- **Quantile** — a generalization of percentile (e.g., the 0.95 quantile is p95).
- **Bucket** — a histogram bin defined by an upper boundary `le=...`.
- **Structured logging** — log lines emitted as JSON (or other parseable format) instead of free text.
- **Request-id** — a unique identifier generated per request and carried through every log line and downstream call for that request.
- **ContextVar** — a Python `contextvars.ContextVar`, used to thread per-request state through async code without passing it as a function argument.
- **ASGI middleware** — middleware in the async server gateway interface protocol used by FastAPI / Starlette.
- **Evaluation harness** — a script that runs against a live service to measure correctness against a labeled fixture.
- **Gold fixture** — a labeled dataset of (input, expected-answer) pairs used to score a model.
- **F1** — the harmonic mean of precision and recall.
- **Precision** — TP / (TP + FP).
- **Recall** — TP / (TP + FN).
- **Micro-average** — aggregate TPs/FPs/FNs across documents, then compute the metric once.
- **Macro-average** — compute the metric per document, then average the metrics.
- **Exact-match** — equality after normalization.
- **Normalization** — a transformation that removes irrelevant variation (whitespace, case) before comparison.
- **Grounding rate** — the fraction of answered questions whose response cites retrieved chunks that actually appeared in the retrieval candidate set.
- **Decline path** — the canonical response a RAG service returns when no grounded answer is possible.
- **Monolith** — a service that wraps multiple responsibilities behind one URL and one deploy pipeline.
- **Microservice** — a service that wraps one responsibility, deployed independently from its peers.
- **Distributed observability** — the practice of correlating metrics, logs, and traces across multiple services.
- **OpenTelemetry** — the open standard for instrumentation across metrics, logs, and traces.
- **Trace** — a causal chain assembled from spans across services.
- **Span** — a single unit of work within a trace (one service handling one operation).

Summary

Three axes of service health — *fast*, *reliable*, *correct* — and three different ways to measure them. Latency lives in a histogram; error rate lives in a counter ratio; quality lives in an evaluation harness. The Module 10 stack is reused unchanged; the new code is three ASGI middlewares (request-id, structured logging, metrics), three metric declarations (counter, histogram, gauge), one `/metrics` mount, and three evaluation scripts that read a labeled fixture and write a Markdown report. The same discipline that put the Pydantic shape on `/extract` puts the counter and histogram on `/metrics` — instrumentation is part of API design, not a deployment afterthought.

The capstone team needs every member to have built each piece. This week ensures that.

What's Next

Core Skills Drill (due tonight after the concept lecture). Add three middlewares and three metrics to a two-endpoint toy FastAPI service (`/echo` and `/sum`), mount `/metrics`, and write a self-test that verifies the metric families are present after a request.

Applied Lab (Tuesday). Add the full observability layer to the M10 backend — three middlewares, three metrics, `/metrics` endpoint, plus a synthetic 3-question RAG smoke evaluator that proves the grounding-check logic you will scale up in the Integration.

Integration Task (Thursday). Author three evaluation harnesses — `eval_ner.py`, `eval_kg.py`, `eval_rag.py` — that run against the live stack, score answers against labeled fixtures, and produce one consolidated Markdown report with three headline numbers plus the derived `/metrics` signals.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.